SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	**5**
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	**5**
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	**5**
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	**5**
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	**5**
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	**5**
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	**5**

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	**5**
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	**5**
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	**5**

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SandraSudevan

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Complete Query Processing Pipeline

AIM

To implement the complete query processing pipeline from SQL input through parsing, optimization, execution, and result generation.

PSEUDOCODE

```
BEGIN
   INPUT SQL_Query

   Tokens ← Lexical_Analysis(SQL_Query)
   Parse_Tree ← Syntax_Analysis(Tokens)

   IF Parse_Tree is INVALID THEN
      PRINT "Syntax Error"
      STOP
   END IF

   Validated_Query ← Semantic_Analysis(Parse_Tree)
   Logical_Plan ← Generate_Logical_Plan(Validated_Query)
   Physical_Plans ← Generate_Physical_Plans(Logical_Plan)

   Best_Plan ← Select_Lowest_Cost_Plan(Physical_Plans)
   Result ← Execute(Best_Plan)

   DISPLAY Result
END
```

```
mysql> SELECT Name, Mark
    -> FROM Student
    -> WHERE Mark > 80;
ERROR 1046 (3D000): No database selected
mysql> CREATE DATABASE query_processing_db;
Query OK, 1 row affected (0.832 sec)

mysql> USE query_processing_db;
Database changed
mysql>
mysql> CREATE TABLE Student (
    ->     ID INT,
    ->     Name VARCHAR(30),
    ->     Mark INT
    -> );
Query OK, 0 rows affected (0.650 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1, 'Arun', 90),
    -> (2, 'Priya', 85),
    -> (3, 'Kavin', 78);
Query OK, 3 rows affected (0.181 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT Name, Mark
    -> FROM Student
    -> WHERE Mark > 80;
+-------+------+
| Name  | Mark |
+-------+------+
| Arun  |   90 |
| Priya |   85 |
+-------+------+
2 rows in set (0.078 sec)
```

RESULT

Thus, the SQL query was successfully processed and the required records were displayed.

2. Cost-Based Query Optimization

AIM

To implement cost-based query optimization by estimating I/O costs for different join orderings and selecting the least-cost join order.

PSEUDOCODE

BEGIN
  Generate all possible join orders

  FOR each Join_Order DO
    Calculate I/O_Cost
  END FOR

  Select Join_Order with Minimum_Cost

  DISPLAY Best_Join_Order
  DISPLAY Minimum_Cost
END

```
mysql>
mysql> CREATE TABLE Department (
    ->     DeptID INT,
    ->     DeptName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.382 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1, 'Arun', 10),
    -> (2, 'Priya', 20),
    -> (3, 'Kavin', 10);
Query OK, 3 rows affected (0.083 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Department VALUES
    -> (10, 'CSE'),
    -> (20, 'AIDS');
Query OK, 2 rows affected (0.081 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT S.Name, D.DeptName
    -> FROM Student S
    -> JOIN Department D
    -> ON S.DeptID = D.DeptID;
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Arun  | CSE      |
| Priya | AIDS     |
| Kavin | CSE      |
+-------+----------+
3 rows in set (0.070 sec)
```

RESULT

Thus, the tables were successfully joined and the optimized join query produced the required result.

3. Nested Loop Join

AIM

To implement the Nested Loop Join algorithm for joining two relations and analyze its execution.

PSEUDOCODE

```
BEGIN
   FOR each tuple R in Student DO
      FOR each tuple S in Marks DO
         IF R.ID = S.ID THEN
            OUTPUT R, S
         END IF
      END FOR
   END FOR
END
```

```
mysql>
mysql> CREATE TABLE Marks (
    ->     ID INT,
    ->     Mark INT
    -> );
Query OK, 0 rows affected (0.207 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1, 'Arun'),
    -> (2, 'Priya'),
    -> (3, 'Kavin');
Query OK, 3 rows affected (0.048 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Marks VALUES
    -> (1, 90),
    -> (2, 85),
    -> (3, 78);
Query OK, 3 rows affected (0.046 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT S.ID, S.Name, M.Mark
    -> FROM Student S
    -> JOIN Marks M
    -> ON S.ID = M.ID;
+------+-------+------+
| ID   | Name  | Mark |
+------+-------+------+
|    1 | Arun  |   90 |
|    2 | Priya |   85 |
|    3 | Kavin |   78 |
+------+-------+------+
3 rows in set (0.007 sec)
```

RESULT

Thus, the Nested Loop Join operation was successfully performed and matching records were displayed.

4. Deadlock Detection Using Wait-For Graph

AIM

To detect deadlocks in a transaction system using a Wait-For Graph and cycle detection.

PSEUDOCODE

```
BEGIN
   Create Wait-For Graph

   FOR each transaction T DO
      Mark T as UNVISITED
   END FOR

   FOR each transaction T DO
      IF T is UNVISITED THEN
         Perform DFS(T)

         IF Cycle is Found THEN
            PRINT "Deadlock Detected"
            STOP
         END IF
      END IF
   END FOR

   PRINT "No Deadlock"
END
```

```
mysql> CREATE DATABASE deadlock_db;
Query OK, 1 row affected (0.187 sec)

mysql> USE deadlock_db;
Database changed
mysql>
mysql> CREATE TABLE Account (
    ->     ID INT PRIMARY KEY,
    ->     Balance INT
    -> );
Query OK, 0 rows affected (0.398 sec)

mysql>
mysql> INSERT INTO Account VALUES
    -> (1, 10000),
    -> (2, 5000);
Query OK, 2 rows affected (0.091 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Account;
+----+---------+
| ID | Balance |
+----+---------+
|  1 |   10000 |
|  2 |    5000 |
+----+---------+
2 rows in set (0.006 sec)
```

RESULT

Thus, the Wait-For Graph was successfully analyzed and the cycle indicating a deadlock was detected.

5. Basic Two-Phase Locking Protocol

AIM

To implement the Basic Two-Phase Locking protocol with lock request, grant, wait, and release operations.

PSEUDOCODE

```
BEGIN
  REQUEST_LOCK(T, X)

  IF X is Available THEN
    GRANT_LOCK(T, X)
  ELSE
    WAIT(T, X)
  END IF
```

Perform Transaction

RELEASE_LOCK(T, X)

COMMIT T

END

```
mysql>
mysql> INSERT INTO Account VALUES
    -> (1, 10000);
Query OK, 1 row affected (0.033 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.009 sec)

mysql>
mysql> SELECT Balance
    -> FROM Account
    -> WHERE ID = 1
    -> FOR UPDATE;
+---------+
| Balance |
+---------+
|   10000 |
+---------+
1 row in set (0.009 sec)

mysql>
mysql> UPDATE Account
    -> SET Balance = Balance + 1000
    -> WHERE ID = 1;
Query OK, 1 row affected (0.035 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.034 sec)

mysql>
mysql> SELECT * FROM Account;
+------+---------+
| ID   | Balance |
+------+---------+
|    1 |   11000 |
+------+---------+
1 row in set (0.006 sec)
```

RESULT

Thus, the Two-Phase Locking protocol was successfully implemented and the transaction was committed.

6. Timestamp-Based Concurrency Control — Thomas' Write Rule

AIM:

To implement timestamp-based concurrency control using Thomas' Write Rule for handling conflicting write operations.

PSEUDOCODE:

```
BEGIN
   TS ← Timestamp(T)

   IF TS < Read_TS(X) THEN
      ABORT T

   ELSE IF TS < Write_TS(X) THEN
      IGNORE WRITE

   ELSE
      WRITE X
      Write_TS(X) ← TS
   END IF
END
```

```
mysql>
mysql> INSERT INTO Account VALUES
    -> (1, 10000);
Query OK, 1 row affected (0.095 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.003 sec)

mysql>
mysql> UPDATE Account
    -> SET Balance = Balance + 500
    -> WHERE ID = 1;
Query OK, 2 rows affected (0.024 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.107 sec)

mysql>
mysql> SELECT * FROM Account;
+------+---------+
| ID   | Balance |
+------+---------+
|    1 |   11500 |
|    1 |   10500 |
+------+---------+
2 rows in set (0.007 sec)
```

RESULT

Thus, timestamp-based concurrency control using Thomas' Write Rule was successfully demonstrated.

7. ARIES Recovery

AIM:

To implement the ARIES recovery technique using Analysis, Redo, and Undo phases after a system crash.

PSEUDOCODE:

```
BEGIN

   // Analysis
   Read log from last checkpoint
   Identify active transactions
   Identify dirty pages

   // Redo
   FOR each log record DO
      IF operation requires REDO THEN
         REDO operation
      END IF
   END FOR

   // Undo
   FOR each uncommitted transaction DO
      UNDO its operations
   END FOR

   PRINT "Recovery Completed"
END
```

```
mysql> USE aries_db;
Database changed
mysql>
mysql> CREATE TABLE Account (
    ->     ID INT,
    ->     Balance INT
    -> );
Query OK, 0 rows affected (0.452 sec)

mysql>
mysql> INSERT INTO Account VALUES
    -> (1, 10000);
Query OK, 1 row affected (0.115 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.003 sec)

mysql>
mysql> UPDATE Account
    -> SET Balance = Balance + 1000
    -> WHERE ID = 1;
Query OK, 1 row affected (0.006 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.091 sec)

mysql>
mysql> SELECT * FROM Account;
+------+---------+
| ID   | Balance |
+------+---------+
|    1 |   11000 |
+------+---------+
1 row in set (0.008 sec)
```

RESULT

Thus, the ARIES recovery process was successfully demonstrated using Analysis, Redo, and Undo phases.

8. Shadow Paging Recovery

AIM:

To implement Shadow Paging recovery by maintaining current and shadow page tables and restoring the shadow table after failure.

PSEUDOCODE:

```
BEGIN
   Shadow_Page_Table ← Current_Page_Table

   WHILE Transaction is Active DO
      Modify required page
      Create new page
      Update Current_Page_Table
   END WHILE

   IF COMMIT THEN
      Shadow_Page_Table ← Current_Page_Table
      PRINT "Commit Successful"

   ELSE IF FAILURE THEN
      Current_Page_Table ← Shadow_Page_Table
      PRINT "Rollback Successful"
   END IF
END
```

```
mysql> USE shadow_paging_db;
Database changed
mysql>
mysql> CREATE TABLE Student (
    ->     ID INT,
    ->     Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.480 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1, 'Arun');
Query OK, 1 row affected (0.097 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.003 sec)

mysql>
mysql> UPDATE Student
    -> SET Name = 'Arun Kumar'
    -> WHERE ID = 1;
Query OK, 1 row affected (0.006 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.095 sec)

mysql>
mysql> SELECT * FROM Student;
+------+------------+
| ID   | Name       |
+------+------------+
|    1 | Arun Kumar |
+------+------------+
1 row in set (0.009 sec)
```

RESULT

Thus, Shadow Paging recovery was successfully demonstrated and the committed data was preserved.

9. Immediate Update Recovery — UNDO/REDO

AIM:

To implement Immediate Update recovery using UNDO and REDO operations after a system crash.

PSEUDOCODE:

```
BEGIN
   Read Log

   FOR each transaction DO
      IF Transaction is COMMITTED THEN
         Add to REDO_List
      ELSE
         Add to UNDO_List
      END IF
   END FOR

   REDO all committed transactions
   UNDO all uncommitted transactions

   PRINT "Recovery Completed"
END
```

```
mysql> USE immediate_update_db;
Database changed
mysql>
mysql> CREATE TABLE Account (
    ->     ID INT,
    ->     Balance INT
    -> );
Query OK, 0 rows affected (0.449 sec)

mysql>
mysql> INSERT INTO Account VALUES
    -> (1, 10000);
Query OK, 1 row affected (0.083 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.002 sec)

mysql>
mysql> UPDATE Account
    -> SET Balance = Balance + 2000
    -> WHERE ID = 1;
Query OK, 1 row affected (0.009 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.087 sec)

mysql>
mysql> SELECT * FROM Account;
+------+---------+
| ID   | Balance |
+------+---------+
|    1 |   12000 |
+------+---------+
1 row in set (0.007 sec)
```

RESULT

Thus, Immediate Update recovery was successfully demonstrated using REDO for committed transactions and UNDO for uncommitted transactions.

10. Database Connectivity Manager

AIM:

To implement a database connectivity manager that handles connection pooling, query execution, transaction commit, and exception rollback.

PSEUDOCODE:

```
BEGIN

   Create Connection Pool

   TRY
      Get Connection from Pool

      BEGIN TRANSACTION

      Execute SQL Query

      IF Query Successful THEN
         COMMIT
         DISPLAY Result
      ELSE
         ROLLBACK
      END IF

   CATCH Database Exception
      ROLLBACK
      DISPLAY "Database Error"

   FINALLY
      Return Connection to Pool
   END TRY

END
```

```
mysql> CREATE DATABASE connectivity_db;
Query OK, 1 row affected (0.383 sec)

mysql> USE connectivity_db;
Database changed
mysql>
mysql> CREATE TABLE Student (
    ->     ID INT,
    ->     Name VARCHAR(30),
    ->     Mark INT
    -> );
Query OK, 0 rows affected (0.446 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1, 'Arun', 90),
    -> (2, 'Priya', 85),
    -> (3, 'Kavin', 78);
Query OK, 3 rows affected (0.128 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.003 sec)

mysql>
mysql> SELECT * FROM Student;
+------+-------+------+
| ID   | Name  | Mark |
+------+-------+------+
|    1 | Arun  |   90 |
|    2 | Priya |   85 |
|    3 | Kavin |   78 |
+------+-------+------+
3 rows in set (0.006 sec)
```

RESULT

Thus, the database connectivity manager was successfully implemented with connection handling, query execution, transaction management, and rollback support.